Name: Natalie Poche
Date: October 16, 2026
UFID: 1533-4992

Report 4 - Embedded Drivers and Bubble Level Application

1 Introduction

This lab explores embedded systems development on the Raspberry Pi RP2040 microcontroller utilizing the Adafruit Feather RP2040 paired with a Propmaker FeatherWing. The primary objective is to design, implement, and integrate modular C++ driver libraries for an onboard LIS3DH I2C accelerometer and a WS2812 RGB NeoMatrix (64 addressable LEDs). Furthermore, an interactive bubble level application (`level.cpp`) is built on top of these drivers to demonstrate real-time sensor data acquisition and hardware actuation.

2 Design

The system architecture divides software responsibilities into three primary components:

1. **LIS3DH Driver (`lis3dh.cpp / .h`):** Interfaces with the LIS3DH accelerometer over I2C1 (SDA on GPIO2, SCL on GPIO3) at 400 kHz. It verifies communication via the `WHO_AM_I` register (expecting `0x33`), configures normal power mode at 1.34 kHz ODR, and sets a $\pm$2g high-resolution range.

2. **NeoMatrix Driver (`neomatrix.cpp / .h`):** Controls the 8×8 WS2812 LED matrix using RP2040 Programmable I/O (PIO) running an asynchronous one-wire protocol (`ws2812.pio`). It also drives GPIO 10 high to enable power on the Propmaker FeatherWing NeoPixel port and handles serpentine pixel mapping.

3. **Bubble Level Application (`level.cpp`):** Continuously samples acceleration values (x, y, z). When both $|x| \leq 0.1$g and $|y| \leq 0.1$g, it illuminates the center 4 LEDs in solid green. Otherwise, it lights up a single red LED indicating the direction of tilt.

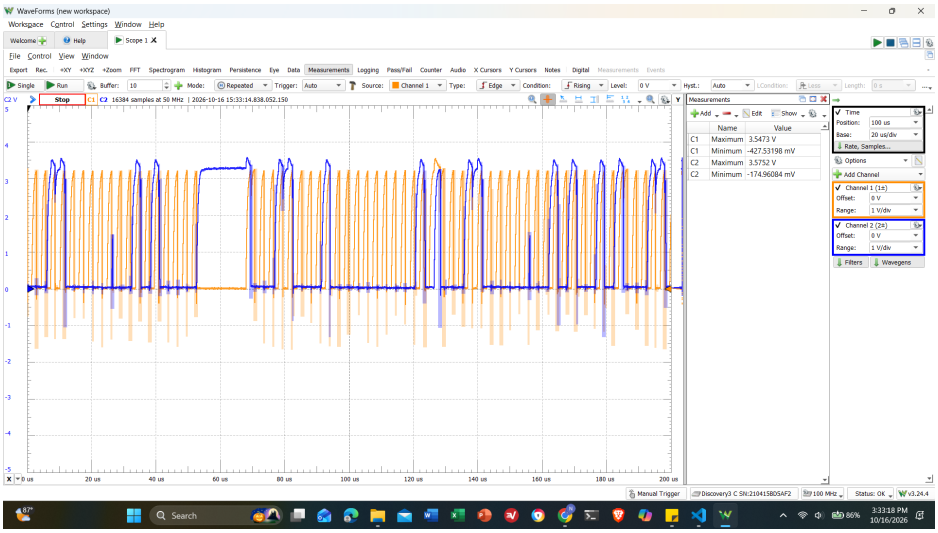


Figure 1: Oscilloscope capture of the I2C bus showing synchronous two-wire serial communication between the RP2040 microcontroller and the LIS3DH accelerometer at a 400 kHz bus clock frequency. Channel 2 (blue) displays the clock line (SCL, GPIO 3) providing regular timing pulses, while Channel 1 (orange) displays the data line (SDA, GPIO 2). The visible bursts correspond to data transactions, such as addressing the accelerometer (`0x18`) and reading/writing internal configuration and data registers (e.g., polling `WHO_AM_I` or pulling axis measurements via `accel.update()`).

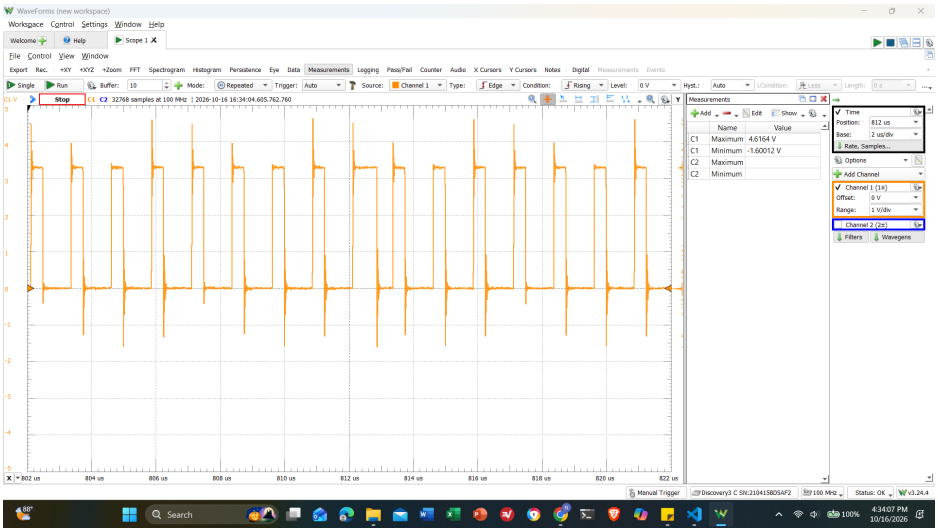


Figure 2: Oscilloscope capture of the NeoMatrix output line implementing a strict single-wire timing protocol used to drive WS2812 RGB LEDs. Generated via the RP2040's Programmable I/O (PIO) state machine on GPIO 7 at 800 kHz, the waveform uses asymmetric pulse-width modulation where each high-frequency pulse encodes binary data based on precise duration timings (T_1, T_2, T_3). The repeated packet trains represent 24-bit GRB color frames shifted out sequentially to update all 64 pixels in the matrix buffer.

2.1 Theoretical Calculations

In high-resolution mode with a ±2g scale, the LIS3DH output data is 16-bit left-justified. The raw register values are shifted right by 6 bits (divided by 64), yielding a sensitivity of 4 mg/digit (0.004 g/digit). The gravitational acceleration along each axis is calculated as:

$$a_x = \text{raw}_x \times 0.004 \text{ g}$$
$$a_y = \text{raw}_y \times 0.004 \text{ g}$$
$$a_z = \text{raw}_z \times 0.004 \text{ g}$$

3 Results and Discussion

The bubble level application was successfully compiled into `level.uf2` using CMake and Ninja, and flashed onto the Adafruit Feather RP2040 via the UF2 bootloader.

Table 1: Comparison of Physical Orientation vs. Expected NeoMatrix Output

Orientation State	Measured x (g)	Measured y (g)	NeoMatrix Output
Flat (Level)	0.02	−0.01	Center 4 LEDs Green ($[3,3], [3,4], [4,3], [4,4]$)
Tilted Right	+0.45	0.05	Single Red LED at Column 7
Tilted Forward	−0.03	−0.52	Single Red LED at Row 0

3.1 Implementation Challenges and Solutions

- **Windows Path Length Limitations:** Nesting projects deep within user directories caused CMake builds of native tools like `pioasm` to fail due to Windows 260-character path limits. *Solution:* Migrated project directories to a short path root (`C:\pico_projects`).
- **Propmaker Power Control:** Initially, the NeoMatrix remained dark despite correct PIO initialization. *Solution:* Configured GPIO 10 as an output and drove it high to enable power on the Propmaker FeatherWing NeoPixel power rail. Also corrected GND and Power from feather to neomatrix as lab instructions say Black is goes to GND and Red goes to 5V, but it's switched.

3.2 Suggestions for Improvement

Structuring the drivers as standalone CMake library targets improves reusability across future embedded projects. Future enhancements could include adding DMA (Direct Memory Access) transfers to the NeoMatrix driver to reduce CPU overhead during pixel buffer writes.

4 Conclusion

The experimental results closely aligned with theoretical expectations (Table **??**). The LIS3DH accelerometer accurately reported tilt vectors, and the WS2812 PIO driver correctly updated the NeoMatrix display. Despite initial build toolchain and path configuration challenges, the project successfully demonstrated embedded driver abstraction and hardware integration on the RP2040 platform.

A Source Code Listings

A.1 README.md

Listing 1: Project README and Build Instructions

````
# RP2040 Embedded Drivers & Bubble Level Application

Developed for **CEN3907C (Computer Engineering)** at the **University of Florida**, this project implements modular C++ hardware drivers for an **LIS3DH I2C Accelerometer** and an **8x8 WS2812 NeoMatrix** display on the **Adafruit Feather RP2040** (paired with an Adafruit Propmaker FeatherWing). The project culminates in an interactive "Bubble Level" application that visualizes tilt dynamics using the LED matrix.

---

## Hardware Architecture & Pinout

* **Microcontroller:** Adafruit Feather RP2040 (RP2040 dual-arm Cortex-M0+)
* **Accelerometer (LIS3DH):** Connected via I2C1 (`SDA = GPIO2`, `SCL = GPIO3`, Address: `0x18`)
* **LED Matrix (NeoMatrix):** WS2812 RGB Matrix driven via PIO (`NEOPIXEL_PIN = GPIO7`, Power Enable: `GPIO10`)

---

## Project Structure

```text
embedded_driver/
          CMakeLists.txt         # CMake build configuration and library targets
          src/
                level.cpp          # Main application logic (Bubble Level state machine)
                lis3dh.cpp         # LIS3DH accelerometer driver implementation
                lis3dh.h           # LIS3DH class definition & register mappings
                neomatrix.cpp      # NeoMatrix driver (PIO management & serpentine mapping)
                neomatrix.h        # NeoMatrix class definition & pixel buffer
                ws2812.pio         # PIO assembly program for WS2812 timing protocol
          README.md              # Project documentation
```

## Instructions

### Initial Build

```
# Create and enter the build directory
mkdir build && cd build

# Configure CMake with automatic SDK fetching enabled
cmake -DPICO_SDK_FETCH_FROM_GIT=ON ..

# Compile the binaries
cmake --build .
```
### Updating after Code Changes

```
# Navigate to the build directory (if not already there)
cd build

````
````

```
# Recompile the project
cmake --build .
```

Cleaning the Build Directory

```
# From the project root directory, remove the build folder and recreate it
rm -rf build && mkdir build && cd build

# Reconfigure and build
cmake -DPICO_SDK_FETCH_FROM_GIT=ON ..
cmake --build .
```
Packing Project for Submission
```
tar --exclude="build" -czvf embedded_driver.tgz embedded_driver
```

A.2 CMakeLists.txt

Listing 2: CMake Configuration File

```
cmake_minimum_required(VERSION 3.13) # Sets the minimum required version of CMake to
    3.13 for building the project

set(PICO_BOARD adafruit_feather_rp2040) # Build for Adafruit Feather PR2040, set before
    SDK is imported)

include(pico_sdk_import.cmake) # Includes the Pico SDK import script to pull in
    Raspberry Pi Pico SDK functions

project(embedded_driver C CXX ASM) # Defines the CMake project name as embedded_driver
    and enables C, CXX, and Assembly languages
set(CMAKE_C_STANDARD 11) # Sets the C standard version to C11
set(CMAKE_CXX_STANDARD 17) # Sets the C++ standard version to C++17

pico_sdk_init() # Initializes the Raspberry Pi Pico SDK environment and build hooks

# add_executable(level # Declares an executable target named level from source files
#     src/level.cpp
#     src/lis3dh.cpp
#     src/neomatrix.cpp
# )

# target_link_libraries(level # Links standard Pico libraries and hardware features to
    the level target
#     pico_stdlib
#     hardware_i2c
#     hardware_pio
#     hardware_gpio
# )

# pico_enable_stdio_usb(level 1) # Enables standard input/output over USB
# pico_enable_stdio_uart(level 0) # Disables standard input/output over UART

# pico_generate_pio_header(level ${CMAKE_CURRENT_LIST_DIR}/src/ws2812.pio) # Generates C
     header from PIO file
# pico_add_extra_outputs(level) # Adds extra binary output formats like uf2, hex, bin

# ---------- LIS3DH accelerometer driver library ----------
```

```
add_library(lis3dh STATIC src/lis3dh.cpp) # Creates a static library target named lis3dh compiled from src/lis3dh.cpp
target_include_directories(lis3dh PUBLIC ${CMAKE_CURRENT_LIST_DIR}/src) # Adds the src directory to the include path for the lis3dh library
target_link_libraries(lis3dh PUBLIC pico_stdlib hardware_i2c) # Links pico_stdlib and hardware_i2c libraries to the lis3dh target

# ---------- NeoMatrix driver library ----------
add_library(neomatrix STATIC src/neomatrix.cpp) # Creates a static library target named neomatrix compiled from src/neomatrix.cpp
# Compile ws2812.pio into ws2812.pio.h, which neomatrix.cpp includes
pico_generate_pio_header(neomatrix ${CMAKE_CURRENT_LIST_DIR}/src/ws2812.pio) # Generates the C header file from the WS2812 PIO program for the neomatrix library
target_include_directories(neomatrix PUBLIC ${CMAKE_CURRENT_LIST_DIR}/src) # Adds the src directory to the include path for the neomatrix library
target_link_libraries(neomatrix PUBLIC pico_stdlib hardware_pio) # Links pico_stdlib and hardware_pio libraries to the neomatrix target

# ---------- Bubble level application ----------
add_executable(level src/level.cpp) # Defines an executable target named level built from src/level.cpp
target_link_libraries(level lis3dh neomatrix pico_stdlib) # Links the custom lis3dh and neomatrix libraries, along with pico_stdlib, to the level executable

# printf goes over USB serial, not UART
pico_enable_stdio_usb(level 1) # Enables standard input/output streaming over USB (stdio via USB CDC)
pico_enable_stdio_uart(level 0) # Disables standard input/output streaming over the hardware UART pins

# Creates level.uf2 (plus .elf, .bin, .hex, .map) in the build directory
pico_add_extra_outputs(level) # Generates UF2 and auxiliary binary file outputs for flashing onto the RP2040 board
```

A.3 src/ws2812.pio

Listing 3: WS2812 PIO Assembly Program

```
;
; Copyright (c) 2020 Raspberry Pi (Trading) Ltd.
;
; SPDX-License-Identifier: BSD-3-Clause
;

.program ws2812
.side_set 1

; The following constants are selected for broad compatibility with WS2812,
; WS2812B, and SK6812 LEDs. Other constants may support higher bandwidths for
; specific LEDs, such as (7,10,8) for WS2812B LEDs.

.define public T1 3
.define public T2 3
.define public T3 4

.lang_opt python sideset_init = pico.PIO.OUT_HIGH
.lang_opt python out_init     = pico.PIO.OUT_HIGH
.lang_opt python out_shiftdir = 1

.wrap_target
bitloop:
```

```
    out x, 1       side 0 [T3 - 1] ; Side-set still takes place when instruction stalls
    jmp !x do_zero side 1 [T1 - 1] ; Branch on the bit we shifted out. Positive pulse
do_one:
    jmp  bitloop   side 1 [T2 - 1] ; Continue driving high, for a long pulse
do_zero:
    nop            side 0 [T2 - 1] ; Or drive low, for a short pulse
.wrap

% c-sdk {
#include "hardware/clocks.h"

static inline void ws2812_program_init(PIO pio, uint sm, uint offset, uint pin, float
    freq, bool rgbw) {

    pio_gpio_init(pio, pin);
    pio_sm_set_consecutive_pindirs(pio, sm, pin, 1, true);

    pio_sm_config c = ws2812_program_get_default_config(offset);
    sm_config_set_sideset_pins(&c, pin);
    sm_config_set_out_shift(&c, false, true, rgbw ? 32 : 24);
    sm_config_set_fifo_join(&c, PIO_FIFO_JOIN_TX);

    int cycles_per_bit = ws2812_T1 + ws2812_T2 + ws2812_T3;
    float div = clock_get_hz(clk_sys) / (freq * cycles_per_bit);
    sm_config_set_clkdiv(&c, div);

    pio_sm_init(pio, sm, offset, &c);
    pio_sm_set_enabled(pio, sm, true);
}
%}
```

A.4 src/level.cpp

Listing 4: Bubble Level Application Main Source Code

```
#include "pico/stdlib.h" // Includes the Raspberry Pi Pico standard library header for
    core functions and time delays
#include "lis3dh.h" // Includes the custom header file for the LIS3DH accelerometer
    driver class
#include "neomatrix.h" // Includes the custom header file for the NeoMatrix LED matrix
    driver class
#include <stdio.h> // Includes the standard C input/output library for printf debugging
#include <math.h> // Includes the standard C math library for mathematical functions
    like fabs or sqrt

#define MATRIX_SIZE     8 // Defines a constant for the width and height of the 8x8 LED
    matrix
#define LEVEL_THRESHOLD 0.1f // Defines the acceleration threshold (in g's) considered
    to be perfectly level
#define GREEN           0x002000  // dimmed // Defines a dimmed green color value in 0
    xRRGGBB format
#define RED             0x200000 // Defines a dimmed red color value in 0xRRGGBB format

// Map an acceleration in [-1 g, +1 g] to an LED index 0..7.
static uint8_t accel_to_index(float a) { // Helper function converting an acceleration
    float value into a matrix coordinate index 0-7
    int i = (int)((a + 1.0f) * (MATRIX_SIZE / 2.0f)); // Scales and shifts the
        acceleration value from [-1, 1] into a pixel grid index range [0, 8]
    if (i < 0) i = 0; // Clamps the calculated index to a minimum of 0 to prevent out-of
        -bounds errors
```

```
    if (i > MATRIX_SIZE - 1) i = MATRIX_SIZE - 1; // Clamps the calculated index to a maximum of 7 (MATRIX_SIZE - 1)
    return (uint8_t)i; // Casts and returns the final constrained index as an 8-bit unsigned integer
}

int main() { // Main program entry point where execution begins
    stdio_init_all(); // Initializes standard input/output types (enabling USB serial output)
    sleep_ms(1000); // Pauses execution for 1000 milliseconds to allow serial connection to stabilize

    LIS3DH accel; // Instantiates an LIS3DH accelerometer driver object named accel

    NeoMatrix matrix(MATRIX_SIZE, MATRIX_SIZE); // Instantiates a NeoMatrix LED grid driver object with 8x8 dimensions

    if (!matrix.init()) { // Attempts to initialize the NeoMatrix hardware and checks if it failed
        printf("NeoMatrix init failed\n"); // Prints an error message to the serial console if matrix initialization fails
    }

    while (!accel.init()) { // Enters a retry loop that runs until the LIS3DH accelerometer successfully initializes
        printf("Failed to initialize LIS3DH accelerometer!\n"); // Prints a failure message over USB serial
        for (uint8_t r = 0; r < MATRIX_SIZE; r++) // Loops through each row of the LED matrix
            for (uint8_t c = 0; c < MATRIX_SIZE; c++) // Loops through each column of the LED matrix
                matrix.set_pixel(r, c, RED); // Fills every pixel on the matrix with the RED color to indicate error state
        matrix.write(); // Transmits the pixel buffer data out to update the physical LEDs
        sleep_ms(1000); // Waits for 1 second before retrying accelerometer initialization
    }

    while (true) { // Enters the main infinite application processing loop
        accel.update(); // Updates acceleration data by reading the latest values from the LIS3DH sensor
        printf("x: %6.3f g  y: %6.3f g  z: %6.3f g\n", accel.x, accel.y, accel.z); // Prints current X, Y, and Z acceleration values to serial
        matrix.clear_pixels(); // Clears all pixels in the matrix buffer (turns them off)

        bool level = (accel.x > -LEVEL_THRESHOLD && accel.x < LEVEL_THRESHOLD && // Evaluates if X acceleration is within the level threshold
                      accel.y > -LEVEL_THRESHOLD && accel.y < LEVEL_THRESHOLD); // Evaluates if Y acceleration is also within the level threshold

        if (level) { // Checks if the device is currently level on both X and Y axes
            // Center 4 LEDs
            matrix.set_pixel(3, 3, GREEN); // Illuminates the top-left center pixel green
            matrix.set_pixel(3, 4, GREEN); // Illuminates the top-right center pixel green
            matrix.set_pixel(4, 3, GREEN); // Illuminates the bottom-left center pixel green
            matrix.set_pixel(4, 4, GREEN); // Illuminates the bottom-right center pixel green
```

```
        } else { // Runs if the device is tilted (not level)
            // x tilt moves the dot left/right, y tilt moves it up/down.
            // If the dot moves opposite to what you expect, negate x or y here.
            uint8_t col = accel_to_index(accel.x); // Maps X acceleration to a matrix column index
            uint8_t row = accel_to_index(accel.y); // Maps Y acceleration to a matrix row index
            matrix.set_pixel(row, col, RED); // Illuminates the corresponding pixel red to indicate tilt direction/position
        }

        matrix.write(); // Sends the updated pixel buffer data to the physical LED hardware
        sleep_ms(50); // Pauses for 50 milliseconds to maintain a stable refresh rate (~20 Hz)
    }
}
```

A.5 src/lis3dh.h

Listing 5: LIS3DH Accelerometer Driver Header

```
#ifndef LIS3DH_H // Header guard start to prevent multiple inclusions of this header file
#define LIS3DH_H

#include <stdint.h> // Includes standard fixed-width integer types (e.g., uint8_t, int16_t)

class LIS3DH { // Declares the LIS3DH accelerometer driver class
public:
    float x; // Public member variable storing the latest X-axis acceleration in g's
    float y; // Public member variable storing the latest Y-axis acceleration in g's
    float z; // Public member variable storing the latest Z-axis acceleration in g's

    LIS3DH(void); // Constructor declaration for initializing class variables
    bool init(void); // Method declaration for initializing the I2C bus and verifying sensor presence
    void set_reg(uint8_t reg, uint8_t val); // Method declaration for writing a value to an internal sensor register
    uint8_t read_reg(uint8_t reg); // Method declaration for reading a single byte from a sensor register
    void update(void); // Method declaration for reading and refreshing all axis measurements

private:
    uint8_t addr; // Private member variable storing the device I2C slave address
    float read_axis(uint8_t reg_low); // Private helper method declaration for reading and converting individual axis data
};

#endif // LIS3DH_H // End of header guard
```

A.6 src/lis3dh.cpp

Listing 6: LIS3DH Accelerometer Driver Implementation

```
#include "lis3dh.h" // Includes the LIS3DH driver class declaration header
#include "hardware/i2c.h" // Includes the Raspberry Pi Pico hardware I2C API header
```

```
#include "hardware/gpio.h" // Includes the Raspberry Pi Pico hardware GPIO configuration header
#include "pico/binary_info.h" // Includes binary information support for the Pico SDK
#include "pico/stdlib.h" // Includes core Pico standard library functions and types

// Hardware connections (Feather RP2040 + Propmaker FeatherWing)
#define LIS3DH_I2C      i2c1    // GPIO2/3 can only be used with I2C1 // Defines the I2C peripheral instance used (i2c1)
#define LIS3DH_SDA_PIN  2 // Defines the GPIO pin number (GPIO 2) used for I2C data (SDA)
#define LIS3DH_SCL_PIN  3 // Defines the GPIO pin number (GPIO 3) used for I2C clock (SCL)
#define LIS3DH_I2C_ADDR 0x18 // Defines the 7-bit I2C device slave address for the LIS3DH accelerometer

#define LIS3DH_REG_WHO_AM_I 0x0F // Defines the register address for the device identification WHO_AM_I register
#define LIS3DH_REG_CTRL1 0x20 // Defines the control register 1 address for setting data rate and power modes
#define LIS3DH_REG_CTRL4 0x23 // Defines the control register 4 address for setting resolution and full-scale range
#define LIS3DH_REG_OUT_X_L 0x28 // Defines the starting data output register address for the X-axis low byte
#define LIS3DH_REG_OUT_Y_L     0x2A // Defines the starting data output register address for the Y-axis low byte
#define LIS3DH_REG_OUT_Z_L     0x2C // Defines the starting data output register address for the Z-axis low byte

#define WHO_AM_I_VALUE  0x33 // Defines the expected chip identification byte value (0x33 for LIS3DH)

LIS3DH::LIS3DH(void) { // Constructor for the LIS3DH class
    x = 0.0f; // Initializes member variable x acceleration to 0.0
    y = 0.0f; // Initializes member variable y acceleration to 0.0
    z = 0.0f; // Initializes member variable z acceleration to 0.0
    addr = LIS3DH_I2C_ADDR; // Assigns the defined I2C address to the class instance variable
}

bool LIS3DH::init(void) { // Initialization method for setting up I2C communication and configuring the sensor
    // Initialize I2C1 at 400kHz. Pins: SDA = GPIO2, SCL = GPIO3 on Feather RP2040
    i2c_init(LIS3DH_I2C, 400 * 1000); // Initializes the I2C peripheral at a 400 kHz bus clock frequency
    gpio_set_function(LIS3DH_SDA_PIN, GPIO_FUNC_I2C); // Configures SDA pin (GPIO 2) to function as an I2C peripheral pin
    gpio_set_function(LIS3DH_SCL_PIN, GPIO_FUNC_I2C); // Configures SCL pin (GPIO 3) to function as an I2C peripheral pin
    gpio_pull_up(LIS3DH_SDA_PIN); // Enables the internal pull-up resistor on the SDA pin
    gpio_pull_up(LIS3DH_SCL_PIN); // Enables the internal pull-up resistor on the SCL pin

    // Verify accelerometer WHO_AM_I register (expects 0x33)
    if (read_reg(LIS3DH_REG_WHO_AM_I) != WHO_AM_I_VALUE) { // Reads WHO_AM_I register and checks if it matches the expected 0x33 value
        return false; // Returns false if the device ID does not match, indicating initialization failure
    }

    // Enable X, Y, Z axes, normal power mode, 1.34 kHz data, rate
```

```
    set_reg(LIS3DH_REG_CTRL1, 0x97); // Configures control register 1 with 0x97 to enable all axes and set output data rate

    // High resolution, 4g range -> CTRL4 = 0x08
    // Blocck date update on, +/-2 g full scale
    set_reg(LIS3DH_REG_CTRL4, 0x08); // Configures control register 4 with 0x08 for scale and block data update settings

    return true; // Returns true indicating successful hardware initialization and configuration
}

void LIS3DH::set_reg(uint8_t reg, uint8_t val) { // Helper method to write a single byte value to a specified register
    uint8_t buf[2] = {reg, val}; // Creates a 2-byte buffer containing the register address and the value to write
    i2c_write_blocking(LIS3DH_I2C, LIS3DH_I2C_ADDR, buf, 2, false); // Performs a blocking I2C write transaction to send the buffer to the device
}

uint8_t LIS3DH::read_reg(uint8_t reg) { // Helper method to read a single byte from a specified register address
    uint8_t val = 0; // Initializes a local variable to store the received register byte
    i2c_write_blocking(LIS3DH_I2C, LIS3DH_I2C_ADDR, &reg, 1, true); // Writes the target register address with a repeated start condition (no stop)
    i2c_read_blocking(LIS3DH_I2C, LIS3DH_I2C_ADDR, &val, 1, false); // Reads 1 byte of data back from the device into val
    return val; // Returns the read register byte value
}

float LIS3DH::read_axis(uint8_t reg_low) { // Helper method to read and convert raw 16-bit data for a single acceleration axis
    uint8_t lsb = read_reg(reg_low); // Reads the low-byte register for the axis
    uint8_t msb = read_reg(reg_low + 1); // Reads the high-byte register for the axis (next consecutive address)
    int16_t raw = (int16_t)((msb << 8) | lsb); // Combines the low and high bytes into a single 16-bit signed integer

    // Data is left-justified 10-bit in normal mode: shift down by 6
    // (divide by 64), then 4 mg per count at +/-2 g full scale.
    return (raw / 64.0f) * 0.004f; // Scales the raw integer value into acceleration in g's
}

void LIS3DH::update(void) { // Method to update all three axis acceleration values from the sensor
    x = read_axis(LIS3DH_REG_OUT_X_L); // Updates the x member variable with the latest X-axis reading
    y = read_axis(LIS3DH_REG_OUT_Y_L); // Updates the y member variable with the latest Y-axis reading
    z = read_axis(LIS3DH_REG_OUT_Z_L); // Updates the z member variable with the latest Z-axis reading
}
```

A.7 src/neomatrix.h

Listing 7: NeoMatrix Display Driver Header

```
#ifndef NEOMATRIX_H // Header guard start to prevent multiple inclusions
#define NEOMATRIX_H

```

```cpp
#include <stdint.h> // Includes fixed-width integer type definitions
#include <vector> // Includes the C++ Standard Template Library vector container header
#include "hardware/pio.h" // Includes the Raspberry Pi Pico PIO hardware library header

// Colors are 0xRRGGBB (0xFF0000 = red)
class NeoMatrix { // Declares the NeoMatrix LED grid driver class
public:
    NeoMatrix(uint8_t width, uint8_t height); // Constructor declaration accepting matrix width and height
    bool init(void); // Powers NeoPixel and starts PIO state machine, returns false if no PIO state machine // Method declaration for hardware initialization
    void set_pixel(uint8_t row, uint8_t col, uint32_t color); // Method declaration for setting an individual pixel color
    void clear_pixels(void); // Sets every pixel off // Method declaration for turning off all pixels in memory
    void write(void); // Send whole buffer to LEDs // Method declaration for transmitting frame buffer to hardware

private:
    uint8_t width; // Private member variable storing matrix width
    uint8_t height; // Private member variable storing matrix height
    std::vector<uint32_t> pixels;   // row-major, width * height entries // Private vector container storing color data for every pixel
    PIO pio; // Private member variable storing the PIO instance block pointer
    uint sm; // Private member variable storing the PIO state machine identifier number
    uint offset; // Private member variable storing the PIO program memory base offset
};

#endif // NEOMATRIX_H // End of header guard
```

A.8 src/neomatrix.cpp

Listing 8: NeoMatrix Display Driver Implementation

```cpp
#include "neomatrix.h" // Includes the NeoMatrix header file
#include "ws2812.pio.h" // Includes the PIO program header for WS2812 LED control
#include "pico/stdlib.h" // Includes core Pico standard library functions and timing macros
#include <string.h> // Includes standard C string/memory manipulation functions
#include "ws2812.pio.h"   // generated from ws2812.pio by CMake // Secondary inclusion of the generated PIO header file

#define NEOPIXEL_PIN 7 // Propmaker "NEO" = Feather D5 = GPIO07 // Defines the GPIO pin connected to the NeoPixel data line (GPIO 7)
#define NEOPIXEL_POWER_PIN 10  // Propmaker "PWR" = FEATHER D10 = GPIO10 // Defines the GPIO pin controlling power to the Propmaker NeoPixels (GPIO 10)
#define WS2812_FREQ         800000 // Defines the target data frequency for WS2812 LEDs (800 kHz)

NeoMatrix::NeoMatrix(uint8_t width, uint8_t height) // Constructor definition taking matrix width and height
    : width(width), // Initializes the width member variable with the constructor parameter
    height(height), // Initializes the height member variable with the constructor parameter
    pixels(width * height, 0), // Initializes the vector storage for pixels with size width*height filled with 0
    pio(pio0), // Initializes the PIO hardware block pointer to pio0
    sm(0), // Initializes the state machine index to 0
    offset(0) {} // Initializes the PIO program memory offset to 0

```

```
19 bool NeoMatrix::init(void) { // Initialization method for configuring power and setting
       up the PIO state machine
20     // Enable power pin for Propmaker FeatherWing NeoPixels
21     // Keeps NeoPixel power off until this pin is driven high
22     gpio_init(NEOPIXEL_POWER_PIN); // Initializes the NeoPixel power control GPIO pin
23     gpio_set_dir(NEOPIXEL_POWER_PIN, GPIO_OUT); // Sets the power pin direction as an
           output
24     gpio_put(NEOPIXEL_POWER_PIN, 1); // Drives the power pin high to turn on power to
           the NeoMatrix LEDs
25     sleep_ms(10); // Pauses for 10 milliseconds to allow power rail voltages to
           stabilize
26
27     // Loads WS2812 program into PIO0 and grab a free state
28     if (!pio_can_add_program(pio, &ws2812_program)) { // Checks if there is enough space
            in PIO memory to load the WS2812 program
29         return false; // Returns false if the program cannot be added to the PIO block
30     }
31     offset = pio_add_program(pio, &ws2812_program); // Loads the WS2812 program into PIO
            instruction memory and saves its offset
32
33     int claimed = pio_claim_unused_sm(pio, false); // Attempts to claim an unused state
           machine on the PIO block without panicking
34     if (claimed < 0) { // Checks if claiming the state machine failed (returned negative
            value)
35         return false; // Returns false if no free state machine is available
36     }
37     sm = (uint)claimed; // Casts and saves the claimed state machine index number
38
39     // RGB leds, not RGBW
40     ws2812_program_init(pio, sm, offset, NEOPIXEL_PIN, WS2812_FREQ, false); //
           Initializes the PIO state machine with the WS2812 driver settings
41
42     clear_pixels(); // Clears the pixel buffer data (turns all pixels off in memory)
43     write(); // Flushes the cleared pixel buffer out to the physical LEDs to ensure a
           clean startup state
44     return true; // Returns true indicating successful matrix initialization
45 }
46
47 void NeoMatrix::set_pixel(uint8_t row, uint8_t col, uint32_t color) { // Method to set a
       specific pixel's color given its row and column
48     if (row >= height || col >= width) return; // Bounds check: returns early if row or
           column exceeds matrix dimensions
49     // Account for serpentine matrix wiring
50     uint8_t actual_col = (row % 2 == 1) ? (width - 1 - col) : col; // Adjusts column
           index for serpentine-wired matrix layouts on odd rows
51     pixels[row * width + actual_col] = color; // Stores the color value into the 1D
           vector at the calculated 2D grid index
52 }
53
54 void NeoMatrix::clear_pixels(void) { // Method to clear all pixels in the frame buffer
55     for (size_t i = 0; i < pixels.size(); i++) { // Loops through every element in the
           pixels vector
56         pixels[i] = 0x00; // Sets each pixel color value to 0 (off)
57     }
58 }
59
60 void NeoMatrix::write(void) { // Method to transmit the pixel buffer to the physical LED
       hardware via PIO
61     for (size_t i = 0; i < pixels.size(); i++) { // Iterates through each pixel in the
           buffer
62         uint32_t color = pixels[i]; // Retrieves the 32-bit color value for the current
               pixel
```

```
63          uint8_t r = (color >> 16) & 0xFF; // Extracts the 8-bit red component from bits
                [23:16]
64          uint8_t g = (color >> 8) & 0xFF; // Extracts the 8-bit green component from bits
                 [15:8]
65          uint8_t b = color & 0xFF; // Extracts the 8-bit blue component from bits [7:0]
66          // WS2812 expects GRB byte ordering
67          uint32_t grb = ((uint32_t)g << 16) | ((uint32_t)r << 8) | b; // Rearranges RGB
                components into GRB byte order expected by WS2812 LEDs
68          pio_sm_put_blocking(pio, sm, grb << 8); // Pushes the shifted GRB color word
                into the PIO state machine transmit FIFO blocking until space is free
69      }
70      sleep_us(300); // Latch duration // Pauses for 300 microseconds to allow LEDs to
            latch the data frame
71  }
```